Consistent Hashing – SDE 1 Interview Guide

A concise interview-focused guide covering the concepts, intuition, proofs, tradeoffs, and common questions related to Consistent Hashing.

1. Why Do We Need Consistent Hashing?

Traditional partitioning uses: server = hash(key) % N.
When N changes (server added/removed), most keys get remapped, causing:
• Massive data migration
• Cache misses
• Increased network traffic

Consistent Hashing minimizes data movement during scaling events.

2. Core Idea

• Hash both servers and keys onto the same circular hash space (hash ring).
• A key belongs to the first server encountered while moving clockwise.
• Only neighboring regions are affected when servers join or leave.

3. What Happens When a Server is Added?

• The new server takes ownership of a portion of its successor's range.
• Only keys in that range move.
• Most keys remain on their existing servers.

4. What Happens When a Server is Removed?

• The removed server's keys move to the next clockwise server.
• No other keys are affected.

5. Why Does Only ~1/N Data Move?

Assume:
• N servers
• K total keys
• Uniform distribution

Each server owns approximately 1/N of the ring.
Therefore each server stores roughly K/N keys.

Adding a server:
New ownership ≈ 1/(N+1)
Keys moved ≈ K/(N+1)
Fraction moved ≈ 1/N.

Removing a server:
Only its K/N keys move.

6. Proof That Each Server Owns ~1/N of the Ring

Let X1, X2, ..., XN be the intervals owned by servers.
X1 + X2 + ... + XN = 1 (entire ring).
All servers are statistically identical.
Therefore:
E[X1] = E[X2] = ... = E[XN] = E[X]

N × E[X] = 1
E[X] = 1/N

Hence, each server owns 1/N of the ring on average.

7. Virtual Nodes (Most Important Follow-Up Question)

Problem:
Servers may not be evenly distributed on the ring.
One server can own significantly more data than others.

Solution:
Place each physical server multiple times on the ring.
These placements are called Virtual Nodes (VNodes).

Benefits:
• Better load balancing
• Smoother scaling
• Reduced hotspots due to uneven server placement

8. Advantages

• Scales easily
• Minimal rebalancing (~1/N keys move)
• Handles node failures gracefully
• Widely used in distributed systems

9. Disadvantages

• More complex than hash(key) % N
• Requires sorted ring lookup (typically O(log N))
• Uneven distribution without virtual nodes
• Hot keys can still overload a server

10. Complexity

Using a sorted map / TreeMap:
• Lookup: O(log N)
• Add server: O(log N)
• Remove server: O(log N)

11. Real-World Use Cases

• Distributed caches
• Distributed databases

• Key-value stores
• CDN routing and large-scale distributed systems

12. Top SDE-1 Interview Questions

1. What problem does Consistent Hashing solve?
2. Why not use hash(key) % N?
3. Explain the hash ring.
4. What happens when a server is added?
5. What happens when a server is removed?
6. Why does only ~1/N data move?
7. What are Virtual Nodes?
8. Why are Virtual Nodes needed?
9. What are the advantages and disadvantages?
10. Where is Consistent Hashing used?

13. 30-Second Interview Summary

Consistent Hashing is a data-partitioning technique used in distributed systems to minimize data movement when servers are added or removed. Servers and keys are mapped onto a hash ring, and each key is assigned to the next clockwise server. Only about 1/N of the data moves during scaling operations, making it far more efficient than traditional modulo-based hashing. Virtual nodes are commonly used to improve load balancing.